Ders 4 Çalışma Özeti

Ders 4 Çalışma Özeti

Genel Konular

- Yapısal kalıplara giriş ve Adapter kalıbı (ekrandan, 00:16:30)
  - Yaratımsal kalıplar bitti; bu derste yapısal (structural) kalıplara geçildi. İlk kalıp Adapter, nispeten basit bir kalıptır.
  - Amaç: bir istekçinin (client) kullanmak istediği sınıfı kullanmasını önleyen tek engel, istekçinin gönderdiği mesajla sınıfın anladığı mesajın ad ve parametre dizilimi (imza) farkıysa, araya istekçinin beklediği imzaya sahip bir adaptör sınıf konur. Adaptör, gelen çağrıyı asıl işi yapacak nesneye iletir.
  - Hocanın benzetmesi: İngiliz prizine Avrupa fişini takmak için kullanılan priz adaptörleri.
  - Kalıp yapısı: `Client` → `Target` (`Request()`); `Adapter`, `Target`'tan türer ve `adaptee` üyesi üzerinden `Adaptee.SpecificRequest()` metodunu çağırır. Hocaya göre `Target` sınıfı kalıptan çıkarılabilir; istekçi doğrudan adaptörü kullanabilir.
- Adapter örneği: çizim programına hazır daire sınıfını uydurmak (ekrandan, 00:20:00 ve 00:23:00)
  - Programda soyut `Shape` (`setLocation()`, `getLocation()`, soyut `display()`, `fill()`, `setColor()`, `undisplay()`) ve alt sınıfları `Point`, `Line`, `Square` vardır.
  - Daha önce yazılmış, kenar yumuşatma (anti-aliasing) dahil her şeyi yapan `XXCircle` sınıfının metot adları farklıdır: `displayIt()`, `fillIt()`, `undisplayIt()`, `setItsColor()`.
  - Çözüm: `Shape`'ten türeyen bir `Circle` adaptör sınıfı yazılır; `private XXCircle myXXCircle` üyesi kurucuda oluşturulur ve `display()` içinde `myXXCircle.displayIt()` çağrılır. Client `Shape`, adaptee `XXCircle`, adaptör `Circle`dır.
- Bridge kalıbı (ekrandan, 00:30:00)
  - Amaç: bir nesnenin soyutlamasını (arayüzünü) gerçekleme ayrıntılarından ayırmak; böylece ikisi bağımsız değişebilir. Hoca bunu Win32 programlamadaki handle kavramına benzetti.
  - Dörtlü Çete örneği: farklı masaüstü ortamlarında (X, Presentation Manager; günümüzde GNOME, KDE) çalışacak bir GUI çerçevesinde pencere. Kalıtımla `XWindow`, `PMWindow`, sonra `XIconWindow`, `PMIconWindow`, `XTransparentWindow` … diye gidilirse her yeni pencere türü ya da her yeni ortam birden fazla sınıf ekletir.
  - Çözüm: soyut `Window` (`DrawText()`, `DrawRect()`) ile soyut `WindowImp` (`DevDrawText()`, `DevDrawLine()`) ayrı hiyerarşilere konur, `Window`'un `imp` üyesi köprüyü kurar. `DrawRect()` dört kez `imp->DevDrawLine()` çağırır. `IconWindow`, `TransientWindow` soyutlama tarafında; `XWindowImp` (`XDrawLine()`, `XDrawString()`), `PMWindowImp` gerçekleme tarafındadır.
  - Yeni bir pencere türü için tek sınıf, yeni bir ortam (örneğin KDE) için yalnız bir `KDEWindowImp` sınıfı eklenir.
  - Kalıp bileşenleri: `Abstraction`, `RefinedAbstraction`, `Implementor`, `ConcreteImplementor`. Abstraction'ın işlemleri isteği Implementor'a yönlendirir; RefinedAbstraction hangi somut gerçekleyiciyle çalıştığını bilmez.
  - Kullanım: soyutlama ile gerçekleme arasındaki bağın kalıcı olması istenmediğinde; hem soyutlamayı hem gerçeklemeyi ayrı özelleşme hiyerarşilerine tabi tutmak istendiğinde.
- Bridge örneği: iki çizim kütüphanesiyle şekil çizmek (ekrandan, 00:40:00, 00:50:00, 00:51:10)
  - İki dikdörtgen ailesi vardır ve her aile ayrı kütüphaneyle çizilir: `DP1.draw_a_line(x1,`

y1, x2, y2) ve DP2.drawline(x1, x2, y1, y2).
  - İlk çözüm: soyut Rectangle (+draw(), #drawLine()), alt sınıflar V1Rectangle (DP1'i kullanır) ve V2Rectangle (DP2'yi kullanır). Daire de eklenince V1Circle, V2Circle gerekir; sınıf sayısı şekil sayısı ile aile sayısının çarpımı kadar artar ($2 \times 2 = 4$, üçüncü şekille $2 \times 3 = 6$).
  - Bridge çözümü: soyut Shape (draw() soyut, #drawLine(), #drawCircle()) ve alt sınıfları Rectangle, Circle; ayrı bir soyut Drawing (drawLine(), drawCircle()) ve alt sınıfları V1Drawing (DP1'i kullanır), V2Drawing (DP2'yi kullanır). Shape'in private Drawing _dp üyesi kurucuda verilir. Yeni bir şekil için yalnız bir sınıf eklenir.
- Composite kalıbı (ekrandan, 01:20:00 ve 01:30:00)
  - Amaç: hiyerarşik parça-bütün ilişkilerini desteklemek; tek tek nesneler ve nesne toplulukları istekçiye aynı biçimde görünür.
  - Dörtlü Çete örneği: grafik çizim programında çizgi, dikdörtgen, metin gibi bileşenler bir resimde gruplanır; bir resim alt resimler içerebilir. Soyut Graphic, alt sınıfları Line, Rectangle, Text ve Picture; Picture.Draw() bütün çocuklarının Draw() metodunu çağırır.
  - Kalıp yapısı: soyut Component (Operation(), Add(Component), Remove(Component), GetChild(int)); Leaf (Operation()); Composite (children listesi, Operation() içinde forall g in children g.Operation();). Ortaya bir ağaç çıkar: kök ve ara düğümler composite, uç düğümler leaf'tir.
  - Farklı türde ara düğümler gerekirse bunlar Composite'in alt sınıfları olarak modellenir; kardeş sınıf yapılırsa çocuk yönetim kodu kopyalanır.
  - Client bir Visitor olabilir (Visitor kalıbı ileride görülecek).
- Composite'te karar verilmesi gereken ayrıntılar
  - Alt düğümün üst düğümünü bilmesi: her bileşende bir üst bileşen başvurusu (myParentComponent) tutulabilir; kök için bu değer null olur. Bununla getRoot() yazılabilir: üst düğüm null değilse onun getRoot() sonucunu döndür, null ise return this.
  - Ara düğüm silme: alt ağacın tamamı silinebilir, çocuğu olan düğümün silinmesi engellenebilir, ya da çocuklar silinen düğümün üst düğümüne aktarılabilir. Üst düğümün bilinmesi bu aktarmayı kolaylaştırır.
- Composite örneği: makine ve parça üretim kataloğu (ekrandan, 01:40:00, 01:48:00, 01:50:00)
  - Slayttaki esin kaynağı: "DP Java Workbook". Makineler ve parçalardan oluşan katalogdan, örneğin belirli bir fabrikada üretilenleri listeleyen raporlar istenir.
  - Soyut ManufacturingItem (machineID, name, factory), somut Machine (parça listesi, addPart, removePart, getParts iterator döndürür) ve MachinePart (parentID). Anlamsız metotlar boş gövdeli ya da null, false, 0 döndüren biçimde yazılır.
  - Report.findItems(items, "Bursa") hem katalogdaki öğelere hem makinelerin parçalarına bakar; derste çalıştırıldığında üç parça buldu (01:53:07).
  - Hoca, aynı yazdırma kodunun iki kez yazıldığını, bunun "duplicate code" kod kusuru olduğunu ve Extract Method ile ayrı bir metoda taşınması gerektiğini gösterdi.
- Decorator kalıbı (ekrandan, 02:10:00, 02:20:00, 02:30:00)
  - Amaç: nesnelere çalışma anında (dinamik olarak) ek sorumluluklar atamak. Kalıtımda özelleşme sınıf düzeyindedir; Decorator'da tek tek nesneler bireysel olarak özelleşir.
  - Örnek: GUI çerçevesinde metin alanı; bazılarına çerçeve (border), bazılarına yatay ya da dikey kaydırma çubuğu, bazılarına ikisi birden eklenir.
  - Çözüm 1: her birleşim için alt sınıf; sınıf sayısı patlar, kod kopyalanır, aynı yetenekler paneller için yeniden yazılır.
  - Çözüm 2: ScrollTextArea ve BorderTextArea'dan çoklu kalıtımla ScrollBorderTextArea. Çoklu kalıtım her dilde desteklenmez ve "diamond problem" ortaya çıkar.
  - Önerilen çözüm: "Sınıfları değil de nesneleri birbirleri ile ilişkilendirelim." Soyut VisualComponent (Draw()), somut TextView; soyut Decorator bir component üyesiyle başka bir VisualComponent'i sarar ve Draw() içinde component->Draw() der; ScrollDecorator (ScrollTo(), scrollPosition) ve BorderDecorator (DrawBorder(), borderWidth) ek yeteneği önce ya da sonra ekler.
  - Kalıp yapısı: Component, ConcreteComponent, soyut Decorator, ConcreteDecoratorA/B.
  - Kullanım: tek tek nesnelere dinamik ve şeffaf olarak (diğerlerini etkilemeden) sorum-

luluk eklemek; bazı sorumlulukları iptal etmek ya da askıya almak; kalıtımın uygun olmadığı anlar. Java'nın stream tabanlı I/O kütüphanesi ve Swing bu kalıbı kullanır.
  - Zayıf yön: yeni sorumluluk yalnız eski sorumluluktan önce ya da sonra eklenebilir ya da eski sorumluluk tamamen iptal edilir (bu durumda nesne zinciri kopabilir). İstekler dekore edilen nesneye değil, en üstteki dekoratöre gönderilmelidir.
- Decorator örneği: satış makbuzuna üst ve alt bilgi eklemek (ekrandan, 02:40:00, 02:46:30, 02:50:00, 03:00:00)
  - `SalesOrder`, makbuzu `SalesTicket` sınıfına bastırır. İstekler adım adım büyür: üst ve alt bilgi, bunların farklı türleri, sonra aynı makbuzda birden çok üst ve alt bilgi.
  - Ara çözüm: soyut `Header` ve `Footer` hiyerarşileri (slayta göre bir Strategy gerçeklemesi). Birden çok üst ve alt bilgi istenince bu çözüm yetmez.
  - Decorator çözümü: soyut `Component` (`prtTicket()`), somut `SalesTicket`, soyut `TicketDecorator` (kurucuda verilen `myComponent`; null değilse `myComponent.prtTicket()`), alt sınıflar `Header1`, `Header2` (önce kendini basar, sonra `super.prtTicket()`) ve `Footer1`, `Footer2` (önce `super.prtTicket()`, sonra kendini basar).
  - `Factory.getComponent()` zincirleme (chaining) ile `new Header1(new Header2(new Footer2(new Footer1(new SalesTicket()))))` döndürür; çıktı `HEADER1, HEADER2, Receipt, FOOTER1, FOOTER2` olur.

Hocanın Özellikle Vurguladığı Kısımlar

- Bridge ile Abstract Factory farkı
  - Abstract Factory'deki ürünler (düğme, radio button, checkbox) birbirinden çok farklıdır. Bridge örneğindeki pencere, simge pencere ve şeffaf pencere ise ortak kodun çoğunu paylaşır; bu yüzden pencere problemi için Abstract Factory uygun değildir.
  - Bridge, Adapter gibi hemen akla gelen bir kalıp değildir; daha çok düşünce ve deneyim ister.
- "Neyin değiştiğini bul ve onu sarmala" ilkesi
  - Birden fazla şey değişiyorsa kalıtım ilişkisini doğru kurmak gerekir. Şekil ve çizim yöntemi tek hiyerarşiye konursa sınıf sayısı çarpımla artar; iki ayrı hiyerarşi ve aralarında köprü ile her yeni şekil ya da yöntem için tek sınıf eklenir.
- `drawLine()` ve `drawCircle()` için erişim belirleyici (ekrandan, 00:40:00 ve 01:12:40)
  - Slayttaki gerekçe: `drawLine` nesnenin iç çalışmasıyla ilgilidir, public olmamalıdır; private metotlar kalıtımla aktarılmaz; protected metotlar başka türden nesnelere görünmez ama alt sınıflara aktarılır.
  - Moladan sonra hoca protected'ın zorunlu olmadığını söyledi: `Shape` soyut olduğu için ondan nesne oluşturulamaz, metot static olmadığı için `Shape.drawLine` diye de çağrılamaz. Amaç alt sınıfların köprü üzerinden çizimi bozmasını engellemekse metot `public final` yapılabilir. Eclipse'te `dp.bridge.solution3` paketinde `public final void drawLine(...)` yazıp derleme hatası olmadığını gösterdi.
- Composite'in zayıf yönü
  - Çocuk yönetim metotları (`Add`, `Remove`, `GetChild`) kalıtımla `Leaf`'e de geçer; bu kalıtımın özüne aykırıdır. Derleme hatası vermemek için Leaf'te boş gövdeli ya da `return false`, `return null` gibi gövdelerle yazılırlar. Hoca bunu esneklik için ödenen, küçük ama rahatsız edici bir bedel olarak gördü.
- Bir parçanın birden fazla makinede bulunması
  - `addPart` eklenen parçanın `parentID`'sinin üzerine yazar. Aynı tür parça birden fazla makinede kullanılıyorsa her makine için ayrı bir `MachinePart` nesnesi oluşturulmalıdır.
- Decorator'da sıralama
  - Header'lar önce kendini basıp sonra iletir, Footer'lar önce iletip sonra kendini basar. Aynı türden birden çok dekoratörün kendi arasındaki çıktı sırası önemliyse ilişkilendirme sırasına dikkat edilmelidir.
- Kalıp adlarını çevirmemek
  - Tasarım kalıplarının önemli bir yararı yazılımcılar arasında ortak bir sözlük oluşturmaktır; bu yüzden hoca kalıp adlarını (Decorator gibi) Türkçeye çevirmeyi tercih etmiyor.
- UML gösterimi üzerine eleştiriler
  - Decorator yapısında `Decorator` ile `component` arasındaki ilişki birebirdir; eski göste-

rimdeki elmas bire çok anlamına gelebileceği için yanıltıcıdır.
  - Makbuz örneğinde `SalesOrder` ile `SalesTicket` arasında üye alan üzerinden sahiplik ilişkisi (düz çizgi) daha doğru bir modellemedir.

Kısa Tekrar Notları

- Adapter: imza uyuşmazlığını çözmek için araya beklenen imzayı taşıyan ve çağrıyı asıl nesneye ileten bir sınıf konur.
- Adapter örneği: `Circle extends Shape`, içinde `XXCircle` üyesi; `display()` → `myXXCircle.displayIt()`.
- Bridge: soyutlama ile gerçekleme ayrı hiyerarşilerdedir, aralarında bir üye alan (köprü) vardır.
- Bridge örneği: `Shape` (Rectangle, Circle) ↔ `Drawing` (V1Drawing → DP1, V2Drawing → DP2).
- Bridge'siz çözümde sınıf sayısı şekil × aile kadar artar.
- Composite: `Component` (ortak arayüz), `Leaf` (çocuğu olmayan uç), `Composite` (çocukları olan ara düğüm, işlemi bütün çocuklarına uygular).
- Composite'in zayıf yönü: çocuk yönetim metotları Leaf'te anlamsızdır ama yazılmak zorundadır.
- Üst düğüm başvurusu `getRoot()` ve düğüm silmeyi kolaylaştırır.
- Decorator: nesneye çalışma anında ek sorumluluk; dekoratör de aynı arayüzü gerçekler ve sardığı nesneye iletir.
- Çoklu kalıtım "diamond problem" doğurur; Java ve C# gibi diller çoklu sınıf kalıtımını desteklemez.
- Decorator'da istek en üstteki dekoratöre gönderilir; dekoratörler zincirleme kurucularla bağlanabilir (Java stream'leri gibi).
- Duplicate code bir kod kusurudur; Extract Method ile giderilir.

Detaylı Açıklamalar

Adapter

Adapter kalıbında üç rol vardır: işi isteyen istekçi, istekçinin beklediği arayüz (Target) ve işi yapabilen ama farklı imzaya sahip sınıf (Adaptee). İstekçinin ya da hazır sınıfın kaynak koduna müdahale edilemiyorsa, Target'ın imzasını taşıyan ve gövdesinde Adaptee'nin metodunu çağıran bir Adapter sınıfı yazılır. Hoca, `Target` soyut sınıfının kalıbın özü olmadığını, istekçinin doğrudan adaptörü bekleyebileceğini söyledi.

Çizim programı örneğinde çerçevenin beklediği `Circle` sınıfı adaptör olur:

```
class Circle extends Shape {
    ...
    private XXCircle myXXCircle;
    ...
    public Circle() {
        myXXCircle = new XXCircle();
    }

    public void display() {
        myXXCircle.displayIt();
    }
    ...
}
```

(ekrandan, 00:23:00) Bütün metotlar yalnız ilgili çağrıyı üye alan üzerinden yürütür; bir değer döndürülecekse `return myXXCircle...` biçiminde iletilir.

> **Ders sonrası not:** Slaytta metot başlığı `void public display()` diye yazılmıştır; Java'da erişim belirleyici dönüş türünden önce gelir, doğrusu `public void display()`dir.

Bridge

Bridge'in anahtar fikri, "soyutlama" ile "gerçekleme"nin ayrı kalıtım hiyerarşilerine konup bir üye alanla birleştirilmesidir. Pencere örneğinde `Window.DrawRect()` dört kez `imp->DevDrawLine()` çağırır; hangi pencere sisteminde çizileceğini `imp`'e verilen somut `WindowImp` belirler. Yeni pencere türü (soyutlama tarafı) ya da yeni pencere sistemi (gerçekleme tarafı) eklemek diğer tarafı etkilemez.

Şekil örneğinde Bridge çözümü şöyledir (ekrandan, 00:51:10):

```
package bridge.solution2;
public abstract class Shape {
    public abstract void draw( );
    private Drawing _dp;
    public Shape (Drawing dp) { _dp= dp; }
    protected void drawLine(double x1, double y1,
        double x2, double y2) { _dp.drawLine(x1, y1, x2, y2); }
    protected void drawCircle(double x,double y,
        double r ) {_dp.drawCircle(x, y, r); }
}
public class Circle extends Shape {
    private double _x, _y, _r;
    public Circle (Drawing dp,
                   double x,double y,double r) {
        super( dp) ;
        _x= x; _y= y; _r= r ;
    }
    public void draw () { drawCircle(_x,_y,_r); }
}
```

`Circle`, `drawCircle`'ı override etmez, yalnız kullanır; hangi kütüphanenin kullanılacağı `Shape`'e verilen `Drawing` nesnesine bağlıdır. `V1Drawing` DP1'in, `V2Drawing` DP2'nin metotlarını çağırır. Hoca, DP1 ve DP2 metotlarının static olduğunu, gerekirse kütüphanelere yeni bir static çizim metodu eklenebileceğini söyledi.

> **Ders sonrası not:** Hoca derste private metotların kalıtımla "aktarıldığını ama doğrudan kullanılamadığını" söyledi. Java dil tanımına göre private üyeler kalıtımla aktarılmaz (alt sınıfın üyesi sayılmaz) ve override edilemez; alt sınıfta aynı imzayla yazılan metot ayrı bir metottur. Slayttaki "private metotlar kalıtım ile aktarılmaz" ifadesi bu anlamda doğrudur.

Composite

Composite'te istekçi yapının kökünü bilirse, kökteki işlem çağrısı bütün ara düğümlere ve yapraklara yayılır. Hoca, düğümlerin üst düğümlerini bilmesi için iki ayrı başvuru (yapraktan ve ara düğümden ayrı ayrı) çizmek yerine bunu `Component` düzeyinde tek bir `myParentComponent` üyesiyle modellemenin yeterli olduğunu gösterdi. Ara düğüm silinirken ne yapılacağı (alt ağacı silmek, silmeyi engellemek, kullanıcıya sormak ya da çocukları üst düğüme aktarmak) iş mantığına göre verilecek bir karardır.

Rapor kodu (ekrandan, 01:48:00):

```
public class Report {
    public static int findItems(ManufacturingItem items[],String factory) {
        int nr = 0;
        System.out.println("Items manufactured in " + factory + ":");
        System.out.println("Nr.\tID\tName");
        System.out.println("---------------------------------");
        for( ManufacturingItem item : items ) {
            if( item==null )continue;
            if( item.getFactory().compareToIgnoreCase(factory) == 0 ) {
                nr++; System.out.println( nr + ".\t" + item.getMachineID()
                        + "\t" + item.getName() );
```

```
            }
            Iterator<ManufacturingItem> subItems = item.getParts();
            if( subItems==null ) continue;
            while( subItems.hasNext() ) {
                ManufacturingItem part = subItems.next();
                if( part.getFactory().compareToIgnoreCase(factory) == 0 ) {
                    nr++; System.out.println( nr + ".\t" +
                            part.getMachineID() + "\t" + part.getName() );
                }
            }
        }
        if( nr == 0 )       System.out.println("No such item found.");
        return nr;
    }
}
```

İki `if` bloğundaki yazdırma kodu aynıdır; hoca bunu duplicate code kusuru olarak işaretledi. Listeleme biçimi değişirse iki yer de değiştirilmek zorundadır; ortak kod ayrı bir static metoda taşınmalıdır.

Decorator

Decorator'da dekoratör de bileşenle aynı soyut türden türediği için, bir bileşenin beklendiği her yerde dekoratör kullanılabilir ve dekoratörler zincirlenebilir. `Header1`'in kodu (ekrandan, 02:46:30):

```
package decorator.example;
public class Header1 extends TicketDecorator {
    public Header1 (Component myComponent) {
        super(myComponent);
    }
    public void prtTicket() {
        // place printing header 1 code here
        System.out.println(getClass().getName());
        super.prtTicket();
    }
}
```

İstemci tarafı (ekrandan, 02:50:00):

```
package decorator.example;
public class Client {
    public static void main(String[] args) {
        Factory myFactory = new Factory();
        Component myComponent = myFactory.getComponent();
        myComponent.prtTicket();
    }
}
public class Factory {
    public Component getComponent () {
        return(new Header1( new Header2( new Footer2(
                new Footer1( new SalesTicket()))))));
    }
}
```

Slayta göre çalışma sonucu `HEADER1, HEADER2, Receipt, FOOTER1, FOOTER2` olur (ekrandan, 03:00:00). Hoca, dekoratörün sardığı nesnenin kurucuyla verilmesini önerdi; Dörtlü Çete bu konuda kesin bir şey söylemez, ama dekoratörün `Operation()` metodunda null denetimi olmadığı için büyük olasılıkla kurucudan verilmesini öngörür.

> **Ders sonrası not:** Hoca slayttaki `Decorator::Draw();` gösteriminin metodun static olduğunu düşündürdüğünü söyledi. C++'ta `Decorator::Draw()` biçimi, alt sınıf için-

den üst sınıfın aynı adlı metodunu çağırmak için kullanılır ve büyük olasılıkla burada Java'daki `super.draw()` anlamındadır; metodun static olduğunu göstermez.

- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.